

TRABAJO PRÁCTICO INTEGRADOR

Documentación Académica y Técnica

Food Store — Sistema de Gestión de Pedidos

Institución	Universidad Tecnológica Nacional (UTN) — Facultad Regional Mendoza
Carrera	Tecnicatura Universitaria en Programación
Materia	Programación II
Integrantes	Federico López Nicolás Morón Ezequiel Menéndez
Proyecto	Food Store — Sistema de Gestión de Pedidos
Entrega	18 de junio de 2026

Índice

Carátula	1
Índice	2
1. Resumen, objetivos y alcance funcional	3
2. Marco teórico	4
3. Arquitectura y organización del proyecto	5
4. Modelo de dominio	6
5. Persistencia y modelo de datos	7
6. Flujo de ejecución y casos de uso	8
7. Validaciones y manejo de errores	9
8. Decisiones técnicas	10
9. Dificultades y soluciones	11
10. Pruebas, estado y limitaciones	12
11. Capturas del sistema	13
12. Instalación y ejecución	14
13. Conclusión, bibliografía y enlaces	15

El informe se elaboró a partir de la revisión del código fuente, el archivo `pom.xml`, el esquema SQL y la ejecución local del proyecto.

1. Resumen, objetivos y alcance

Resumen

Food Store es una aplicación de consola desarrollada en Java para administrar la operación básica de un local gastronómico. El sistema gestiona categorías, productos, usuarios y pedidos, y persiste la información en una base de datos relacional MySQL mediante JDBC.

Objetivo general

Integrar en un proyecto funcional los contenidos de Programación II: Programación Orientada a Objetos, herencia, interfaces, colecciones, enumeraciones, excepciones, arquitectura por capas, acceso a datos y diseño relacional.

Objetivos específicos

- Modelar entidades y relaciones del dominio.
- Separar presentación, negocio y persistencia.
- Aplicar validaciones antes de persistir.
- Utilizar consultas SQL parametrizadas.
- Implementar operaciones CRUD.
- Calcular el total de cada pedido.
- Conservar historial mediante baja lógica.
- Gestionar dependencias con Maven.

Alcance funcional

Módulo	Operaciones disponibles
Categorías	Listar, crear, editar y eliminar lógicamente.
Productos	Listar, crear, editar y eliminar; asociar a una categoría.
Usuarios	Listar, crear, editar y eliminar; validar correo único.
Pedidos	Listar, crear, agregar detalles, actualizar estado y forma de pago, y eliminar.

El sistema es monousuario y se ejecuta por consola. No incluye interfaz gráfica, API web ni autenticación.

2. Marco teórico

Java y POO

Java es un lenguaje orientado a objetos, tipado y multiplataforma. El proyecto aplica encapsulamiento, herencia, interfaces, polimorfismo básico, colecciones y sobrescritura de métodos.

JDBC

JDBC es la API estándar para acceder desde Java a bases relacionales. Se utilizan Connection, PreparedStatement, ResultSet y DriverManager.

MySQL

MySQL almacena información estructurada en tablas relacionadas. Las claves primarias identifican registros y las claves foráneas garantizan relaciones entre entidades.

Maven

Maven configura Java 21, administra MySQL Connector/J y ejecuta las fases de limpieza, compilación, prueba y empaquetado definidas por el ciclo de construcción.

Patrón DAO

El patrón Data Access Object concentra SQL y mapeo relacional. De esta forma, los menús y servicios no dependen directamente de consultas concretas.

Baja lógica

En vez de borrar registros físicamente, se actualiza el atributo eliminado. Esto permite conservar referencias y datos históricos.

Conceptos aplicados

- **Herencia:** las entidades principales extienden la clase abstracta Base.
- **Interfaz:** Pedido implementa Calculable.
- **Enumeraciones:** valores cerrados para rol, estado y forma de pago.
- **Excepciones:** errores de dominio expresados mediante clases específicas.
- **Try-with-resources:** cierre automático de conexiones, sentencias y resultados.

3. Arquitectura y organización



Capa / paquete	Responsabilidad	Componentes
Presentación	Leer opciones y mostrar resultados.	Main, clases Menu*
service	Coordinar casos de uso y aplicar validaciones.	Cuatro clases *Service
dao	Ejecutar SQL y mapear filas a objetos.	Clases CRUD*
entities	Representar el modelo y su comportamiento.	Seis entidades e interfaz
config	Centralizar y comprobar la conexión.	DatabaseConnection, TestConnection
enums	Restringir valores válidos.	Rol, Estado, FormaPago
exceptions	Representar errores del dominio.	Tres excepciones personalizadas

Flujo de dependencias

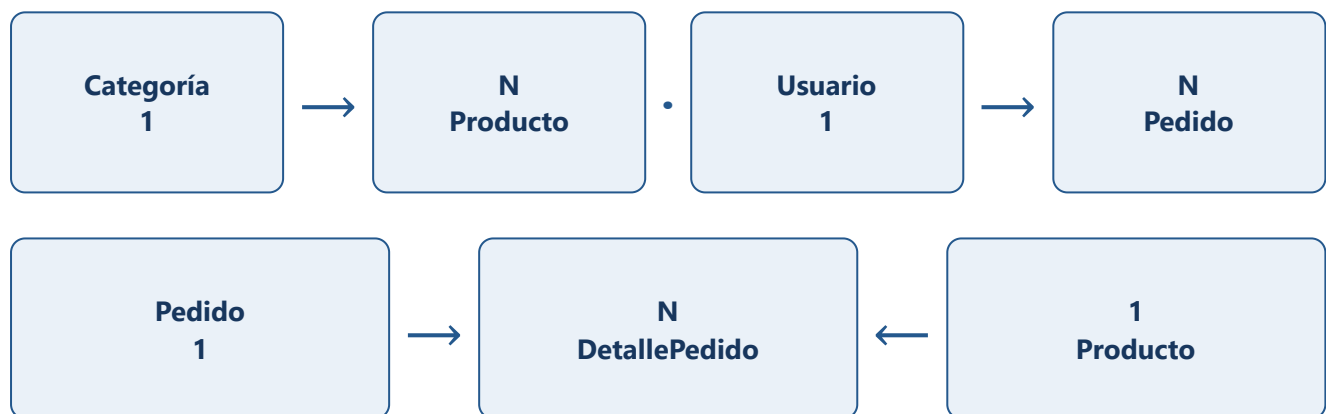
Los menús conocen a los servicios; los servicios conocen a los DAO y entidades; los DAO conocen la configuración JDBC y las entidades. La base de datos no es accedida directamente desde la presentación.

```
src/  
├─ Main.java y Menu*.java  
├─ config/  
├─ dao/  
├─ entities/  
├─ enums/  
├─ exceptions/  
└─ service/
```

4. Modelo de dominio

Base (abstracta) id, eliminado, createdAt equals(), hashCode()	Usuario nombre, apellido, mail, celular, contrasenia, rol
Categoria nombre, descripcion	Pedido implements Calculable fecha, estado, total, formaPago, usuario, detalles
Producto nombre, precio, descripcion, stock, imagen, disponible, categoria	DetallePedido cantidad, subtotal, producto

Relaciones



Comportamiento relevante

- `Pedido.addDetallePedido()` crea el detalle y solicita el recálculo.
- `Pedido.calcularTotal()` suma los subtotales de la colección.
- `Base.equals()` compara entidades persistidas por clase e identificador.
- Los métodos `toString()` generan representaciones para la consola.

5. Persistencia y modelo de datos

La aplicación utiliza una base de datos MySQL ya creada y configurada para el proyecto, por lo que no requiere una instalación local en el entorno habitual del equipo. El archivo `schema.sql` documenta su estructura y permite reproducirla en otro entorno. Todas las tablas incluyen identificador, fecha de creación y marca de baja lógica.

Tabla	Datos principales	Relaciones
categoria	nombre, descripción	1:N con producto
producto	precio, stock, imagen, disponible	FK a categoría
usuario	datos personales, mail único, rol	1:N con pedido
pedido	fecha, estado, total, forma de pago	FK a usuario
detalle_pedido	cantidad y subtotal	FK a pedido y producto

Esquema relacional simplificado

```
CATEGORIA (id PK)
  1 ——— N PRODUCTO (categoria_id FK)

USUARIO (id PK)
  1 ——— N PEDIDO (usuario_id FK)
          |
          1
          |
          N
          DETALLE_PEDIDO
          ├── pedido_id FK
          └── producto_id FK — N:1 PRODUCTO
```

Estrategia JDBC

- Consultas parametrizadas con `PreparedStatement`.
- Obtención de identificadores con `RETURN_GENERATED_KEYS`.
- Mapeo manual desde `ResultSet` a entidades.
- Consultas con `JOIN` para reconstruir relaciones.
- Cierre automático mediante `try-with-resources`.

6. Flujo de ejecución y casos de uso

Flujo general

1. Main crea el Scanner y los cuatro servicios.
2. El usuario selecciona categorías, productos, usuarios o pedidos.
3. La clase de menú solicita y convierte los datos.
4. El servicio valida y coordina la operación.
5. El DAO ejecuta SQL y devuelve objetos o identificadores.
6. El resultado se informa por consola.

Caso de uso: crear producto



Caso de uso: crear pedido



Operaciones CRUD

Operación	Implementación
Create	INSERT y recuperación de clave generada.
Read	SELECT filtrando registros activos.
Update	UPDATE por ID y <code>eliminado = FALSE</code> .
Delete	UPDATE <code>eliminado = TRUE</code> ; no se ejecuta borrado físico.

7. Validaciones y manejo de errores

Regla	Capa	Respuesta
Precio no negativo	ProductoService	StockInvalidoException
Stock no negativo	ProductoService	StockInvalidoException
Cantidad mayor que cero	PedidoService	StockInvalidoException
Stock suficiente	PedidoService	StockInvalidoException
Correo obligatorio y único	UsuarioService	MailDuplicadoException o mensaje
Entidad activa existente	Servicios	EntidadNoEncontradaException
Entrada numérica válida	Menús	Captura de NumberFormatException

Excepciones personalizadas

EntidadNoEncontradaException

Indica que un ID no existe o corresponde a un registro eliminado.

MailDuplicadoException

Impide crear o editar un usuario con un correo ya utilizado.

StockInvalidoException

Expresa valores inválidos de precio, stock o cantidad solicitada.

SQLException

Se captura en servicios para informar fallos de persistencia con contexto.

Gestión de recursos

Los DAO utilizan try-with-resources, por lo que conexiones, sentencias y resultados se cierran aunque ocurra una excepción. Antes de eliminar, los menús solicitan confirmación explícita.

8. Decisiones técnicas

Decisión	Justificación	Impacto
Aplicación de consola	La consigna se centra en POO y persistencia.	Interfaz simple y menor complejidad accidental.
Arquitectura por capas	Separa interacción, negocio y SQL.	Mayor mantenibilidad y responsabilidades claras.
Patrón DAO	Evita consultas SQL en menús y entidades.	Persistencia localizada y reutilizable.
Clase abstracta Base	Las entidades comparten ID, fecha y baja lógica.	Reduce duplicación y unifica identidad.
Enums	Roles, estados y pagos poseen valores limitados.	Evita cadenas inconsistentes en el dominio.
Baja lógica	Se requiere conservar relaciones e historial.	Los listados filtran eliminado = FALSE.
PreparedStatement	Separa SQL de valores ingresados.	Mejora seguridad y evita problemas de formato.
JDBC sin ORM	Permite practicar SQL y acceso a datos directo.	Mapeo explícito, adecuado al tamaño del proyecto.
Maven	Centraliza versión de Java y dependencias.	Compilación reproducible mediante comandos estándar.

La solución evita incorporar frameworks web, ORM o contenedores de inyección porque no aportan valor necesario al alcance académico actual.

Consistencia con SOLID

La separación de menús, servicios, entidades y DAO aplica principalmente responsabilidad única. La interfaz calculable define un contrato pequeño y específico. Para una evolución futura, convendría inyectar los DAO mediante interfaces para reducir el acoplamiento de los servicios.

9. Dificultades y soluciones

Mapeo entre objetos y tablas

Dificultad: reconstruir productos, usuarios y pedidos junto con sus relaciones. **Solución:** consultas con JOIN y mapeo manual de cada fila a entidades Java.

Recuperación de identificadores

Dificultad: continuar el flujo después de insertar una entidad. **Solución:** utilizar `Statement.RETURN_GENERATED_KEYS` y asignar el ID devuelto al objeto.

Conservación del historial

Dificultad: eliminar registros sin romper relaciones. **Solución:** adoptar baja lógica y filtrar registros activos en búsquedas y listados.

Separación de responsabilidades

Dificultad: evitar que los menús contengan SQL y reglas de negocio. **Solución:** organizar el proyecto en presentación, servicios, DAO, dominio y configuración.

Consistencia de un pedido

Dificultad: una compra afecta cabecera, detalles, total y stock. **Estado actual:** las operaciones están separadas. **Mejora necesaria:** agruparlas en una única transacción con `commit` y `rollback`.

Las dificultades y soluciones fueron redactadas a partir de la estructura implementada y del historial del repositorio, sin atribuir procesos que no pueden verificarse en el código.

10. Pruebas, estado y limitaciones

Verificación realizada — 18 de junio de 2026

Prueba	Resultado
mvn clean package	BUILD SUCCESS
Compilación	28 archivos fuente; destino Java 21.
Empaquetado	Generado target/mi-proyecto-1.0-SNAPSHOT.jar.
config.TestConnection	Conexión MySQL establecida.
Main y submenús	Ejecución correcta.
Base de datos	Conexión JDBC verificada.
Pruebas automatizadas	No existen fuentes en src/test.

Limitaciones verificadas

1. Pedido y detalles no se persisten dentro de una transacción común.
2. El total calculado no se actualiza en la tabla pedido.
3. La carga de detalles utiliza el subtotal como precio unitario y puede recalcular mal el total.
4. Se valida el stock, pero no se descuenta al registrar el detalle.
5. Las credenciales están dentro del código fuente.
6. La conexión devuelve null ante fallos y puede provocar errores secundarios.

Mejoras prioritarias

Implementar transacciones JDBC; actualizar total y stock de forma atómica; extraer configuración a variables de entorno; usar `BigDecimal` para importes; agregar pruebas unitarias y de integración.

11. Capturas del sistema

Capturas generadas a partir de la ejecución real del proyecto compilado. Se muestran operaciones de navegación que no requieren modificar datos.



```
Ejecucion de Food Store

PS N:\TP1 Prog2\TPI-Programacion2-Lopez-Moron-Menendez> java ... Main

=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione: _
```

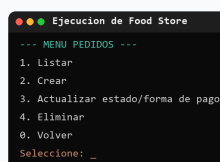
Figura 1. Menú principal con acceso a las cuatro áreas funcionales.



```
Ejecucion de Food Store

--- MENU PRODUCTOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: _
```

Figura 2. Operaciones CRUD disponibles para productos.



```
Ejecucion de Food Store

--- MENU PEDIDOS ---
1. Listar
2. Crear
3. Actualizar estado/forma de pago
4. Eliminar
0. Volver
Seleccione: _
```

Figura 3. Creación, consulta, actualización y baja de pedidos.

Las capturas muestran la navegación principal de la aplicación conectada mediante la configuración JDBC del proyecto.

12. Instalación y ejecución

Requisitos

- JDK 21 o superior.
- Apache Maven 3.9 o superior.
- Acceso a la base de datos MySQL configurada.

Base de datos

El equipo utiliza una base MySQL ya provisionada. Para ejecutar el sistema se requiere acceso a esa base y una configuración JDBC válida. `schema.sql` queda disponible como referencia y respaldo de la estructura.

Configurar la conexión

```
private static final String URL =  
    "jdbc:mysql://localhost:3306/pedidos_db";  
private static final String USER = "TU_USUARIO";  
private static final String PASSWORD = "TU_CONTRASENA";
```

Las credenciales reales deben mantenerse fuera de commits públicos.

Compilar y descargar dependencias

```
mvn clean package  
mvn dependency:copy-dependencies -DoutputDirectory=target/dependency
```

Ejecutar

```
# Windows  
java -cp "target/classes;target/dependency/*" Main  
  
# Linux / macOS  
java -cp "target/classes:target/dependency/*" Main
```

Para comprobar únicamente la conexión, ejecutar la clase `config.TestConnection`. Desde IntelliJ IDEA se puede iniciar directamente la clase `Main`.

13. Conclusión, bibliografía y enlaces

Conclusión

Food Store integra correctamente los conceptos centrales de Programación II en una aplicación de consola persistente. La separación entre menús, servicios, DAO y entidades permite comprender el flujo del sistema y mantener responsabilidades definidas. Las operaciones CRUD principales están implementadas, Maven compila el proyecto y la conexión con MySQL fue verificada.

El siguiente paso técnico prioritario es garantizar la consistencia del agregado pedido: cabecera, detalles, total y stock deben actualizarse dentro de una transacción. También resulta necesario agregar pruebas automatizadas y externalizar las credenciales.

Bibliografía / Webgrafía

1. Oracle. *Java SE 21 Documentation*.
<https://docs.oracle.com/en/java/javase/21/>
2. Oracle. *JDBC Basics*.
<https://docs.oracle.com/javase/tutorial/jdbc/basics/>
3. Apache Software Foundation. *Maven Documentation*.
<https://maven.apache.org/guides/>
4. Oracle. *MySQL Reference Manual*.
<https://dev.mysql.com/doc/>
5. Oracle. *MySQL Connector/J Developer Guide*.
<https://dev.mysql.com/doc/connector-j/en/>

Enlaces de entrega

Repositorio:

github.com/EzequielMenendez/TPI-Programacion2-Lopez-Moron-Menendez